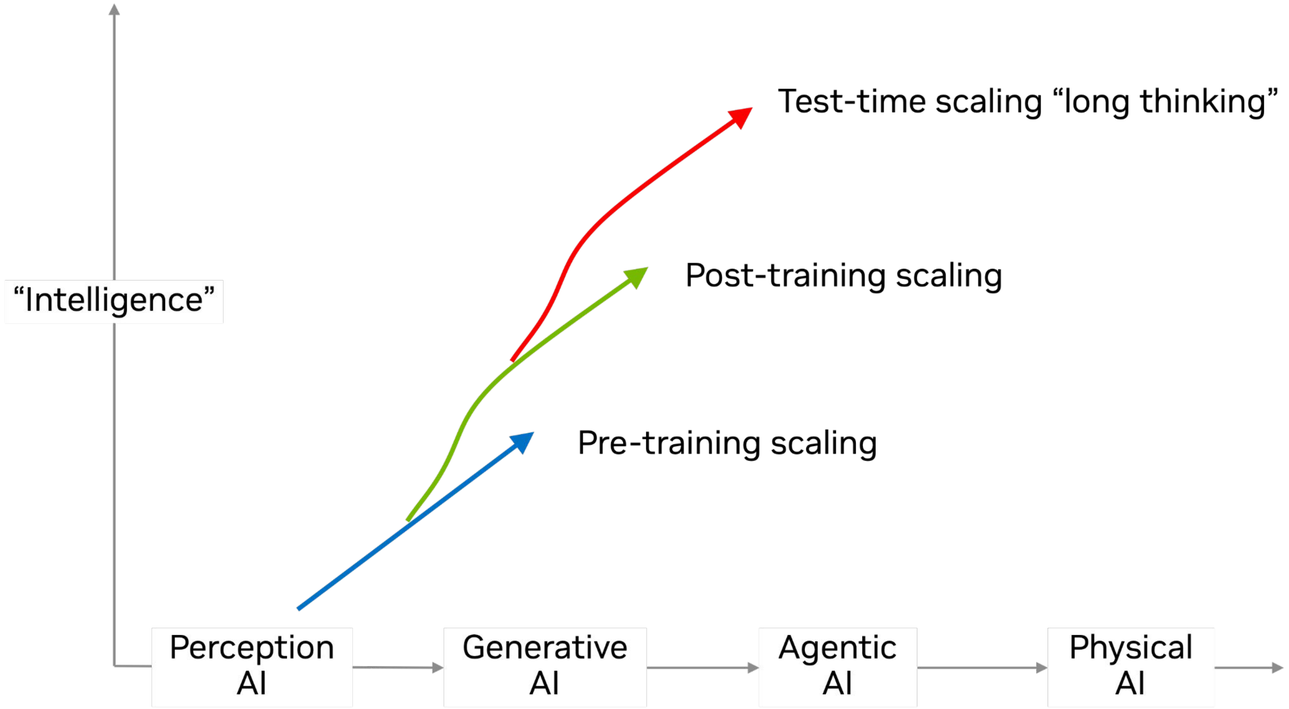
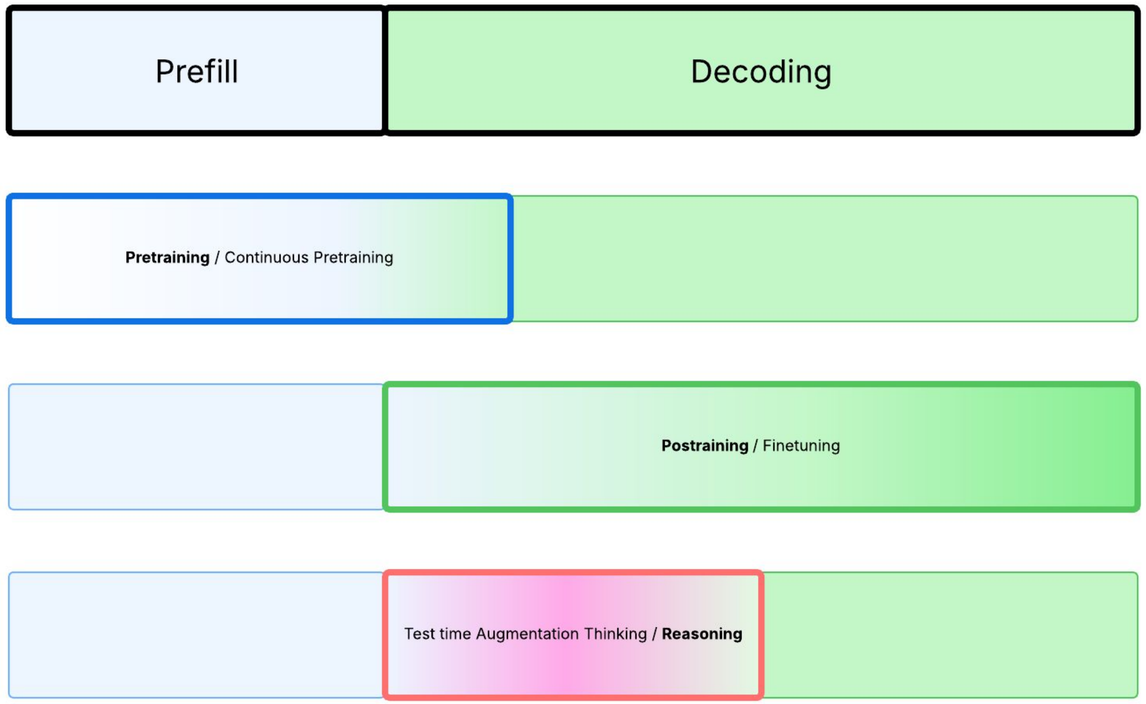
NeMo Curator Master Deck

Quan Tran Minh, PhD,
Senior Developer Relations Manager (DevRel), NVIDIA

Three Scaling Laws





Getting Started

```
# Python package manager
uv pip install "nemo-curator[text_cuda13]"




# Pull the container
docker pull nvcr.io/nvidia/nemo-curator:26.02

# Run the container
docker run --gpus all -it --rm nvcr.io/nvidia/nemo-curator:26.02


mkdir -p ~/nemo_curator/data/sample
mkdir -p ~/nemo_curator/data/curated
```

```
from nemo_curator.core.client import RayClient
```

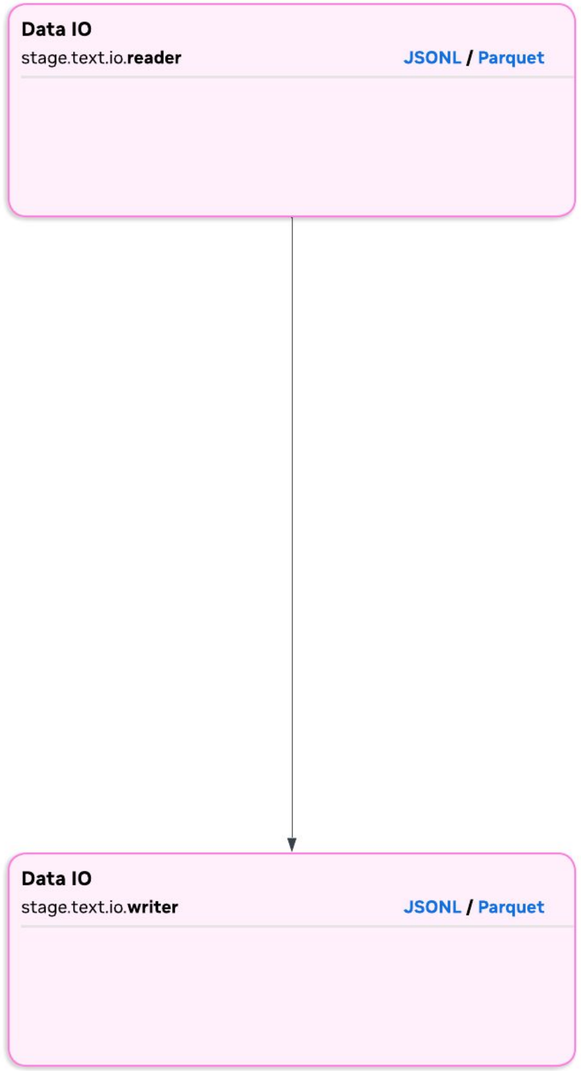
Create a pipeline for Ray Cluster

```
def main():
    # Initialize Ray client
    ray_client = RayClient()
    ray_client.start()

    # Put Nemo Curator Pipeline and Stages HERE



    # Stop Ray client
    ray_client.stop()


if __name__ == "__main__":
    main()
```

Data IO

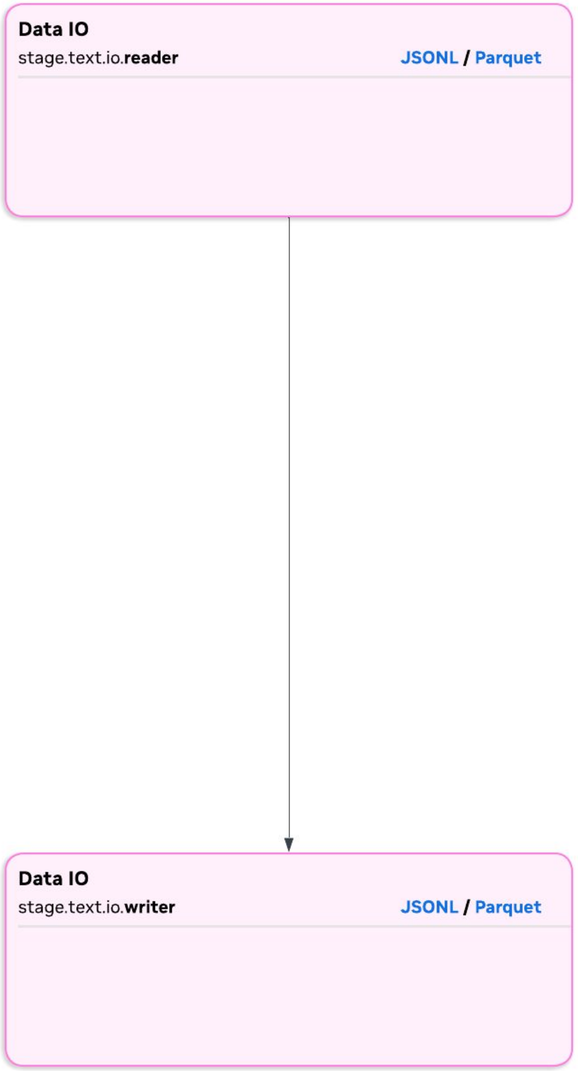
Data IO Overview

JSONL



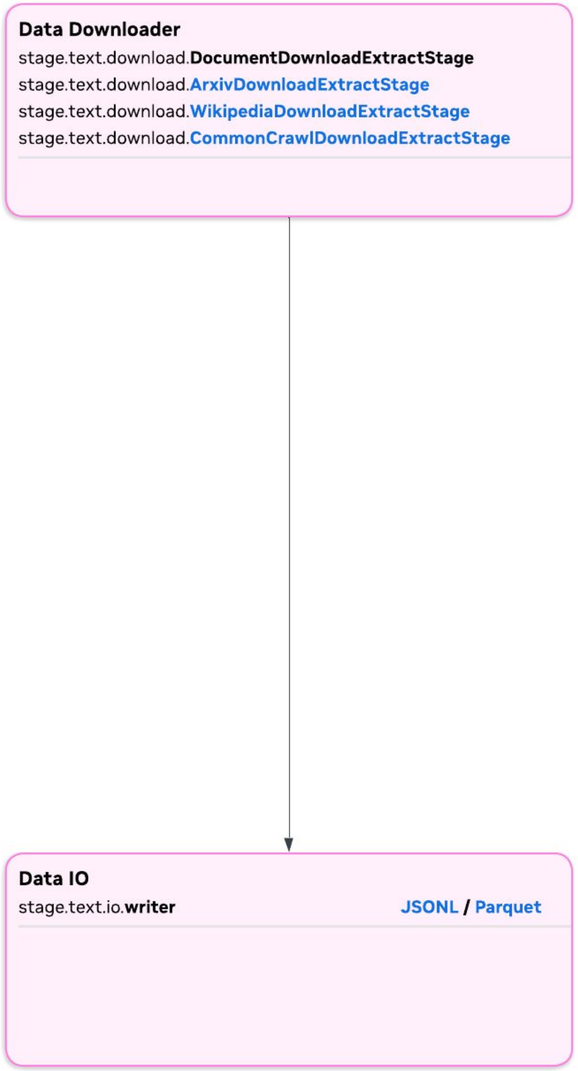

```python
from nemo_curator.pipeline import Pipeline
from nemo_curator.stages.text.io.reader import JsonlReader
from nemo_curator.stages.text.io.writer import JsonlWriter

# Create a pipeline for text curation
pipeline = Pipeline(
    name="text_curation_pipeline",
    description="Basic text quality filtering pipeline"
)

# Add stages to the pipeline
pipeline.add_stage(
    JsonlReader(
        file_paths="~/nemo_curator/data/sample/",
        files_per_partition=4,
        fields=["text", "id"]
    )
)

### Declare Modify, Classify, Filter, Deduplicate HERE

# Write the curated results
pipeline.add_stage(
    JsonlWriter("~/nemo_curator/data/curated")
)
# Execute the pipeline
results = pipeline.run()

print(f"Pipeline completed successfully!")
print(f"Processed {len(results) if results else 0} tasks.")
```

Parquet



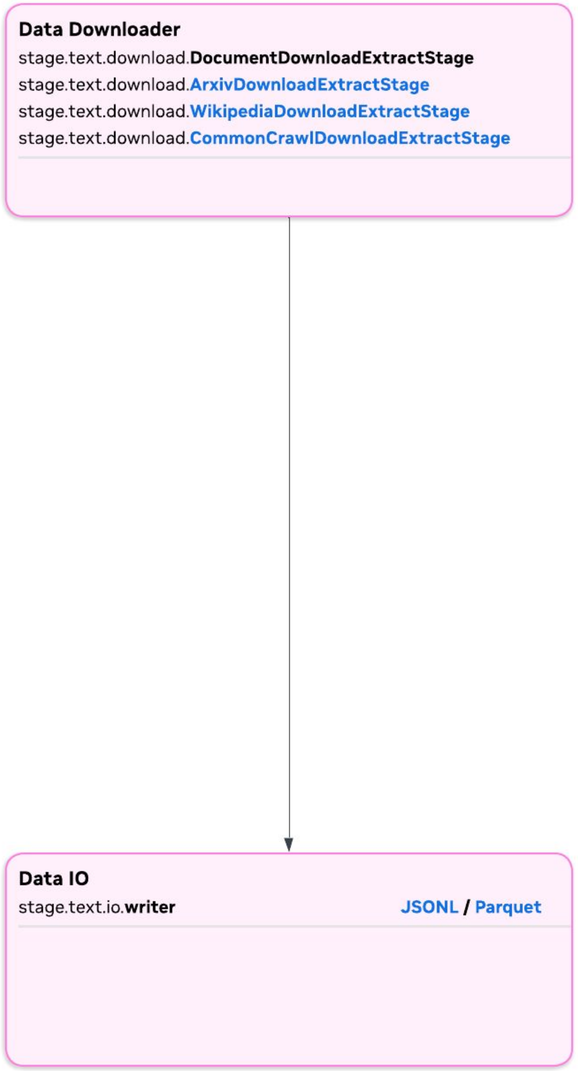

```python
from nemo_curator.pipeline import Pipeline
from nemo_curator.stages.text.io.reader import ParquetReader
from nemo_curator.stages.text.io.writer import ParquetWriter

# Create a pipeline for text curation
pipeline = Pipeline(
    name="text_curation_pipeline",
    description="Basic text quality filtering pipeline"
)

# Add stages to the pipeline
pipeline.add_stage(
    ParquetReader(
        file_paths="~/nemo_curator/data/sample/",
        files_per_partition=4,
        fields=["text", "id"]
    )
)

### Declare Modify, Classify, Filter, Deduplicate HERE

# Write the curated results
pipeline.add_stage(
    ParquetWriter("~/nemo_curator/data/curated")
)
# Execute the pipeline
results = pipeline.run()

print(f"Pipeline completed successfully!”)
print(f"Processed {len(results) if results else 0} tasks.")
```

Arxiv



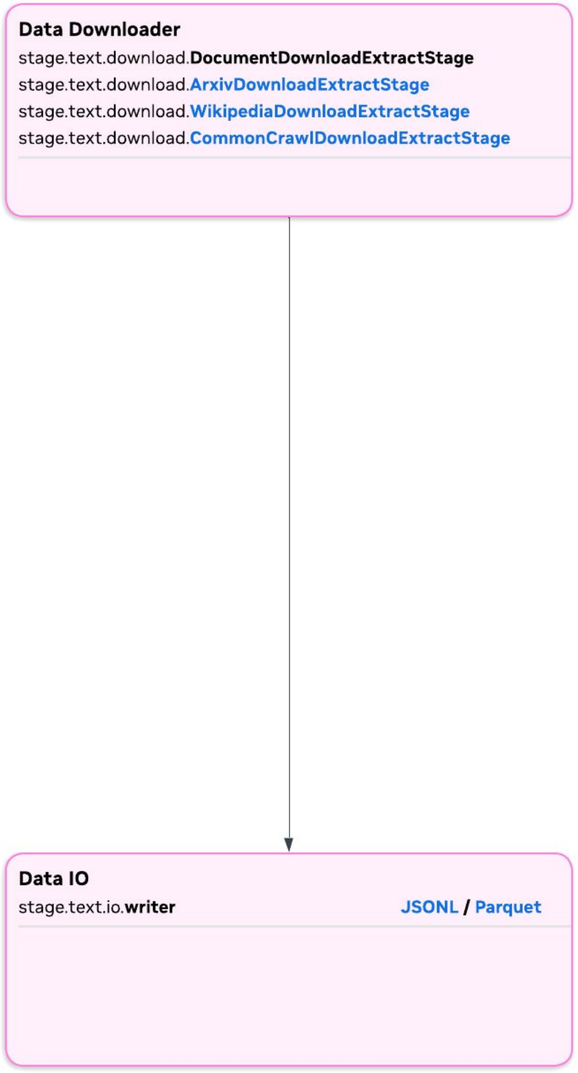

```python
from nemo_curator.pipeline import Pipeline
from nemo_curator.stages.text.download import ArxivDownloadExtractStage
from nemo_curator.stages.text.io.writer import JsonlWriter

# Create a pipeline for text curation
pipeline = Pipeline(
    name="arxiv_pipeline",
    description="Download and process ArXiv LaTeX sources"
)

# Add ArXiv stage
pipeline.add_stage(
    ArxivDownloadExtractStage(
        download_dir="./arxiv_downloads",
        url_limit=5,        # optional: number of tar files to process
        record_limit=1000,  # optional: max papers per tar
        add_filename_column=True,
        verbose=True,
    )
)
### Declare Modify, Classify, Filter, Deduplicate HERE
# Write the curated results
pipeline.add_stage(
    JsonlWriter("~./arxiv_output")
)
# Execute the pipeline
results = pipeline.run()

print(f"Pipeline completed successfully!")
print(f"Processed {len(results) if results else 0} tasks.")
```

Wikipedia



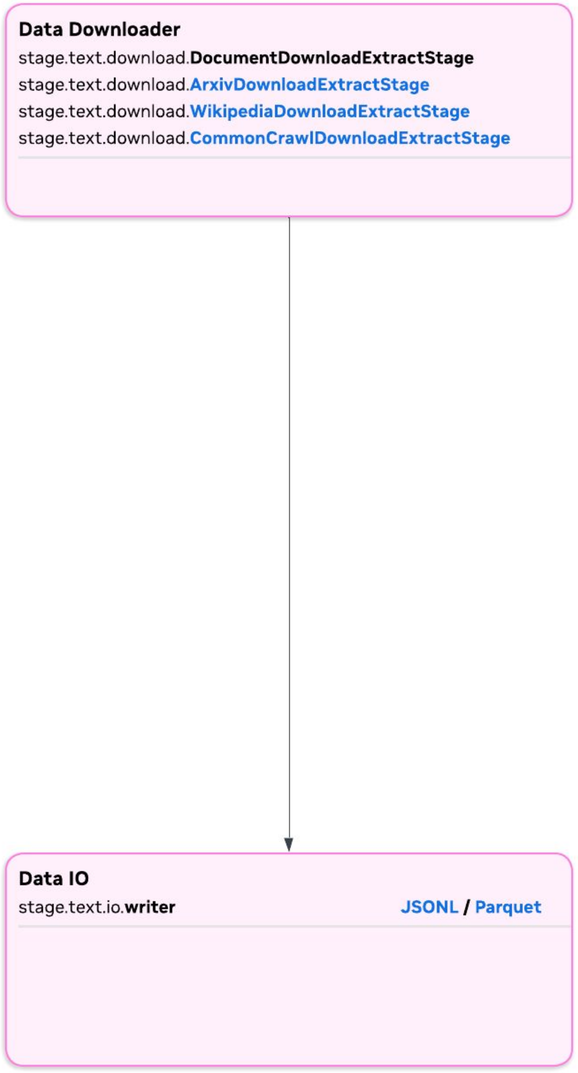

```python
from nemo_curator.pipeline import Pipeline
from nemo_curator.stages.text.download import WikipediaDownloadExtractStage
from nemo_curator.stages.text.io.writer import JsonlWriter

# Create a pipeline for text curation
pipeline = Pipeline(
    name="wikipedia_pipeline",
    description="Download and process Wikipedia dumps"
)

# Add Wikipedia stage
pipeline.add_stage(
    WikipediaDownloadExtractStage(
        download_dir="./wikipedia_downloads",
        language="en",
        dump_date=None,          # None uses latest dump
        url_limit=5,             # Optional: limit number of dump files
        record_limit=1000,       # Optional: limit articles per dump file
    )
)
### Declare Modify, Classify, Filter, Deduplicate HERE
# Write the curated results
pipeline.add_stage(
    JsonlWriter("~./wikipedia_output")
)
# Execute the pipeline
results = pipeline.run()

print(f"Pipeline completed successfully!”)
print(f"Processed {len(results) if results else 0} tasks.")
```

CommonCrawl




```python
from nemo_curator.pipeline import Pipeline
from nemo_curator.stages.text.download import CommonCrawlDownloadExtractStage
from nemo_curator.stages.text.io.writer import JsonlWriter

# Create a pipeline for text curation
pipeline = Pipeline(
    name="common_crawl_pipeline",
    description="Download and process Common Crawl data"
)

# Add CommonCrawl stage
pipeline.add_stage(
    CommonCrawlDownloadExtractStage(
        download_dir="./cc_downloads",
        crawl_type="main",        # or "news"
        use_aws_to_download=True, # Faster S3 downloads (requires s5cmd)
        url_limit=10,             # Limit number of WARC files for testing
        record_limit=1000,        # Limit records per WARC file
    )
)
### Declare Modify, Classify, Filter, Deduplicate HERE
# Write the curated results
pipeline.add_stage(
    JsonlWriter("~./cc_output")
)
# Execute the pipeline
results = pipeline.run()

print(f"Pipeline completed successfully!”)
print(f"Processed {len(results) if results else 0} tasks.")
```

Custom Data

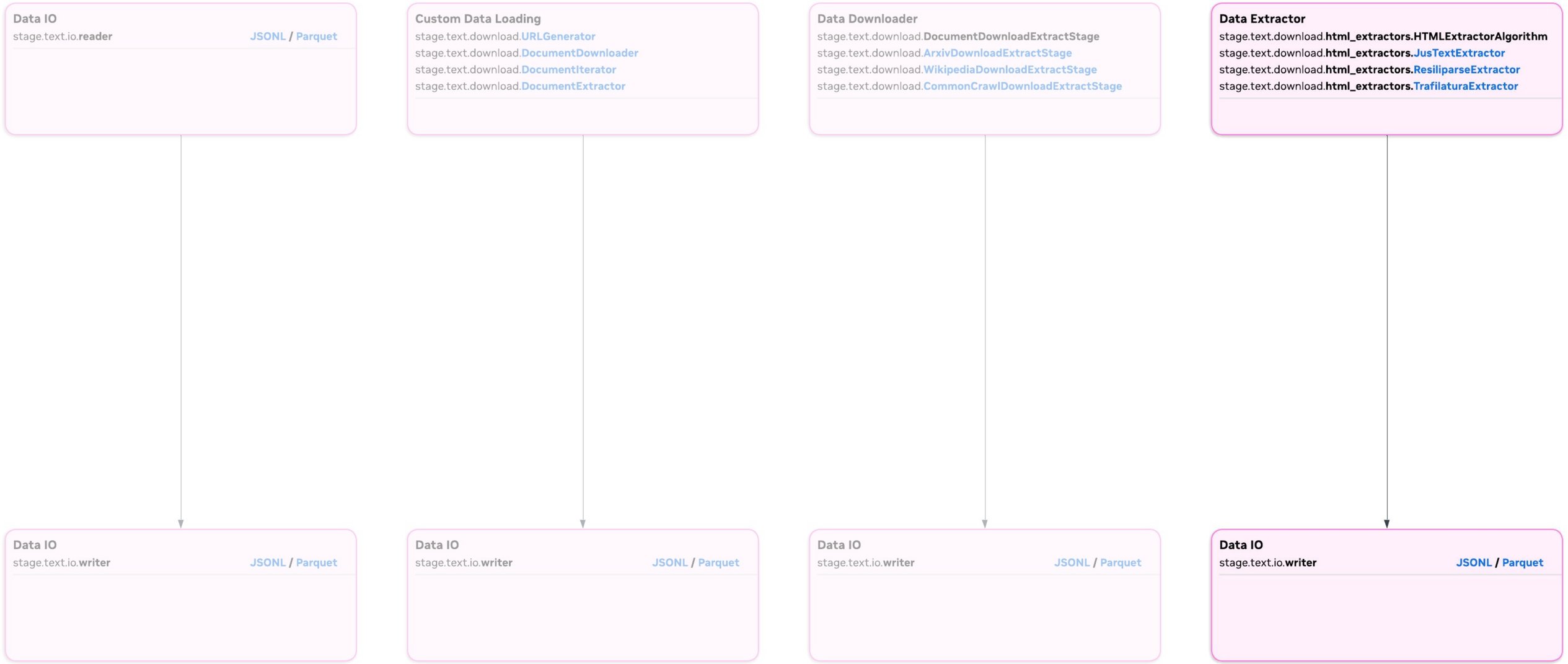



```python
from nemo_curator.pipeline import Pipeline
from custom_data_source import CustomDataStage
from nemo_curator.stages.text.io.writer import JsonlWriter

# Create a pipeline for text curation
pipeline = Pipeline(
    name="custom_data_pipeline",
    description="Load and process custom dataset"
)

# Add Custom Data Downloader stage
pipeline.add_stage(
    CustomDataStage(
        #
        #
        #
        #
        #
    )
)
### Declare Modify, Classify, Filter, Deduplicate HERE
# Write the curated results
pipeline.add_stage(
    JsonlWriter("~./output")
)
# Execute the pipeline
results = pipeline.run()

print(f"Pipeline completed successfully!")
print(f"Processed {len(results) if results else 0} tasks.")
```

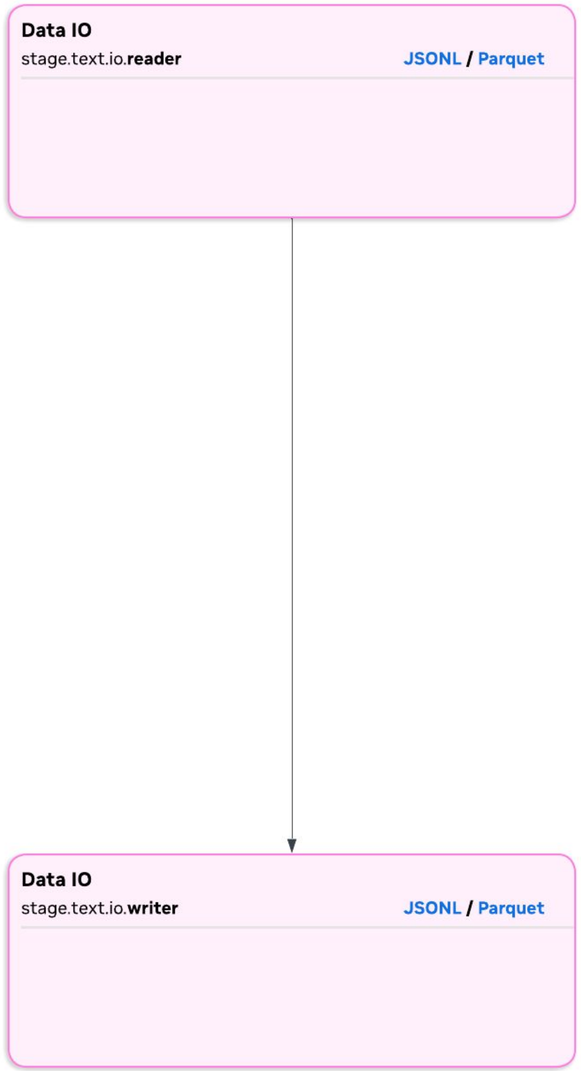
```
custom_data_source/
├── __init__.py
├── stage.py            # Main composite stage
├── url_generation.py   # URL generation logic
├── download.py         # Download implementation
├── iterator.py         # File iteration logic
└── extract.py          # Data extraction logic
```

NVIDIA

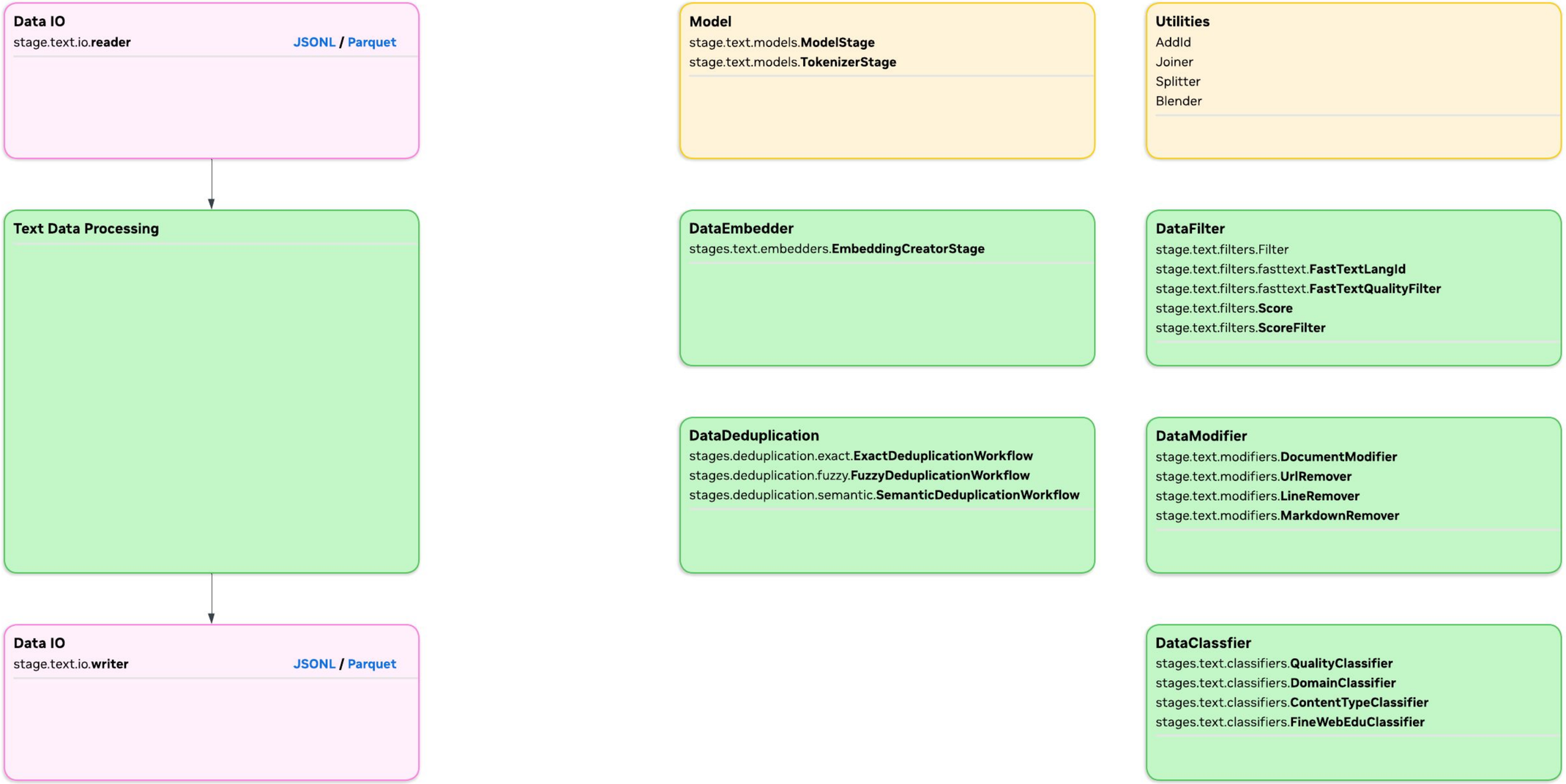
HMTL Data Extractor



Nemo Curator Pipeline

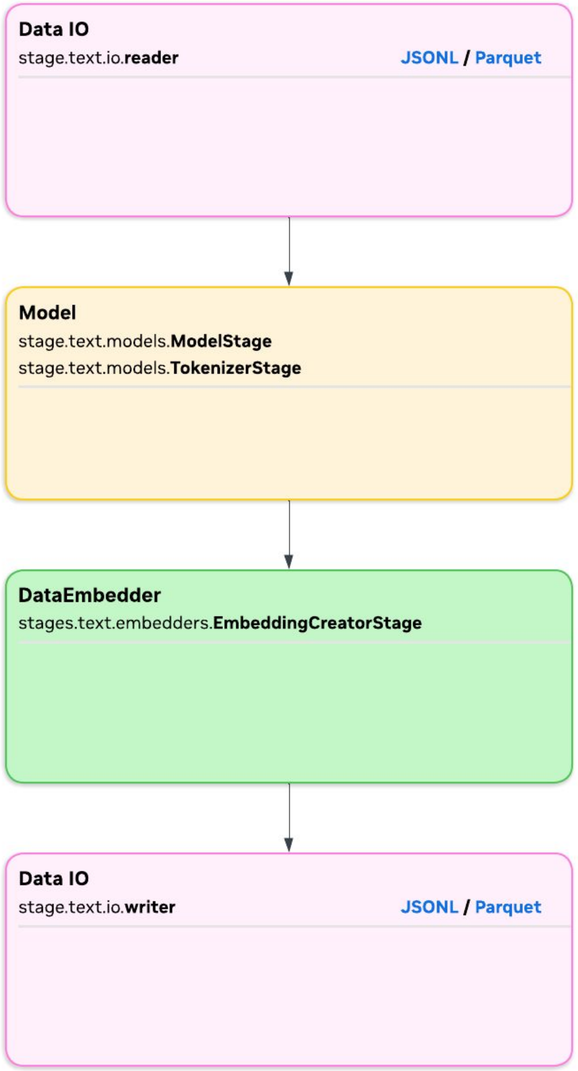


Nemo Curator Pipeline



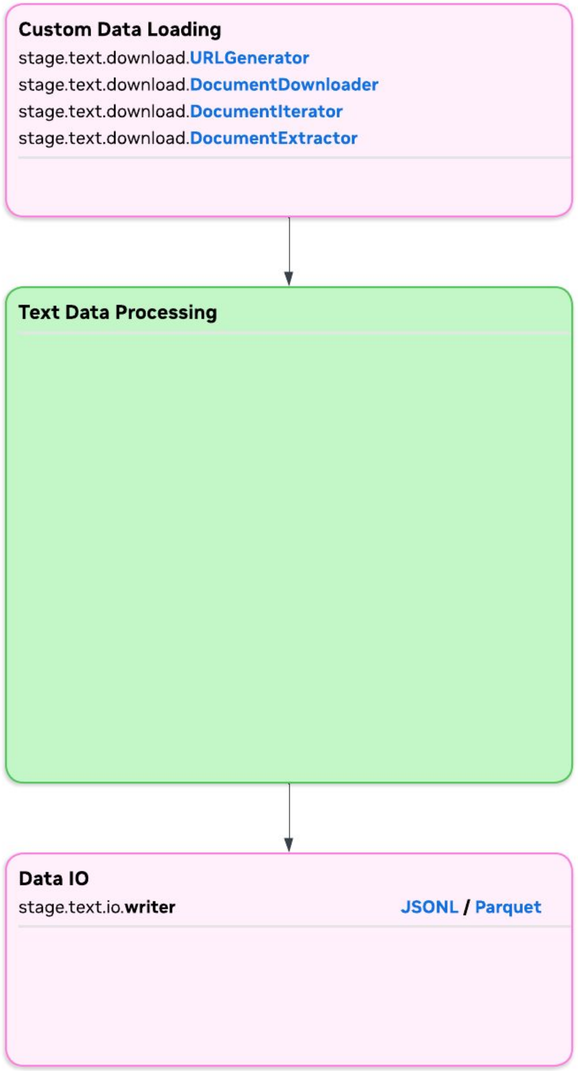
Advance Customization

Embedding Pipeline



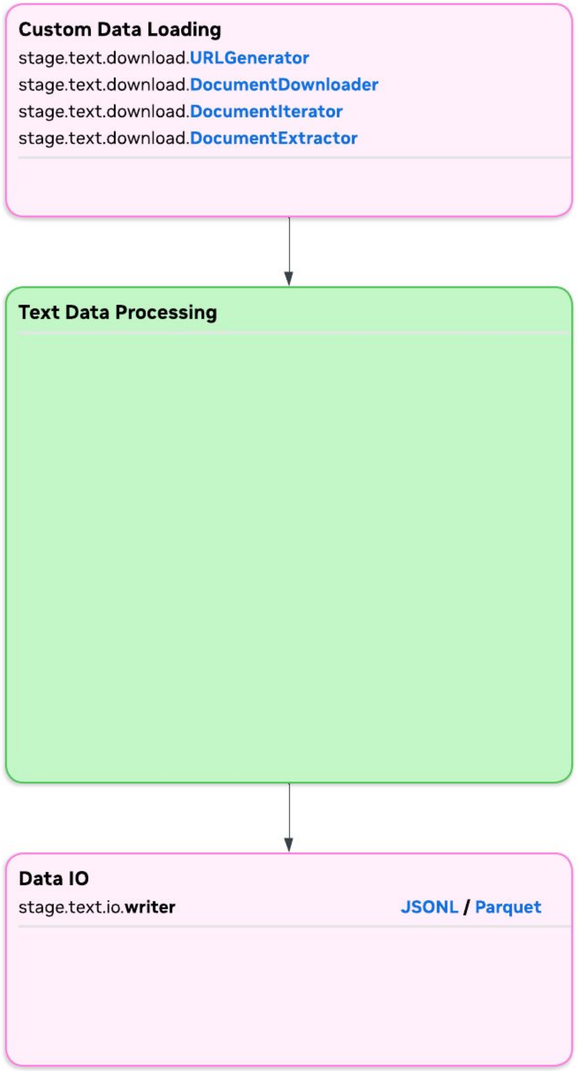

```
pipeline = Pipeline(
    name="embedding_generation_pipeline",
    stages=[
        ParquetReader(
            file_paths=input_files,
            files_per_partition=files_per_partition,
            fields=["text"],
            _generate_ids=False,
        ),
        EmbeddingCreatorStage(
            model_identifier=model_identifier,
            text_field="text",
            max_seq_length=max_seq_length,
            max_chars=None,
            embedding_pooling="mean_pooling",
            model_inference_batch_size=batch_size,
        ),
        ParquetWriter(
            path=output_path,
            fields=["embeddings"],
        ),
    ],
)
```

Advanced Customization



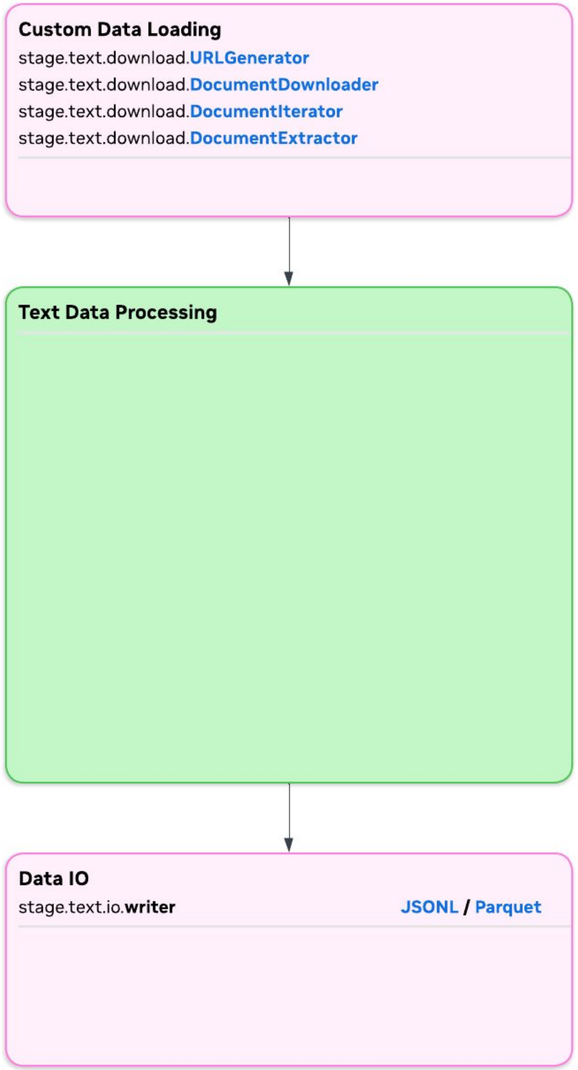

```python
from nemo_curator.stages.text.download import URLGenerator

class YourURLGenerator(URLGenerator):
    def generate_urls(self) -> list[str]:
        # Return list of URLs to download
        return ["https://example.com/file1.zip", ...]
```

Advanced Customization



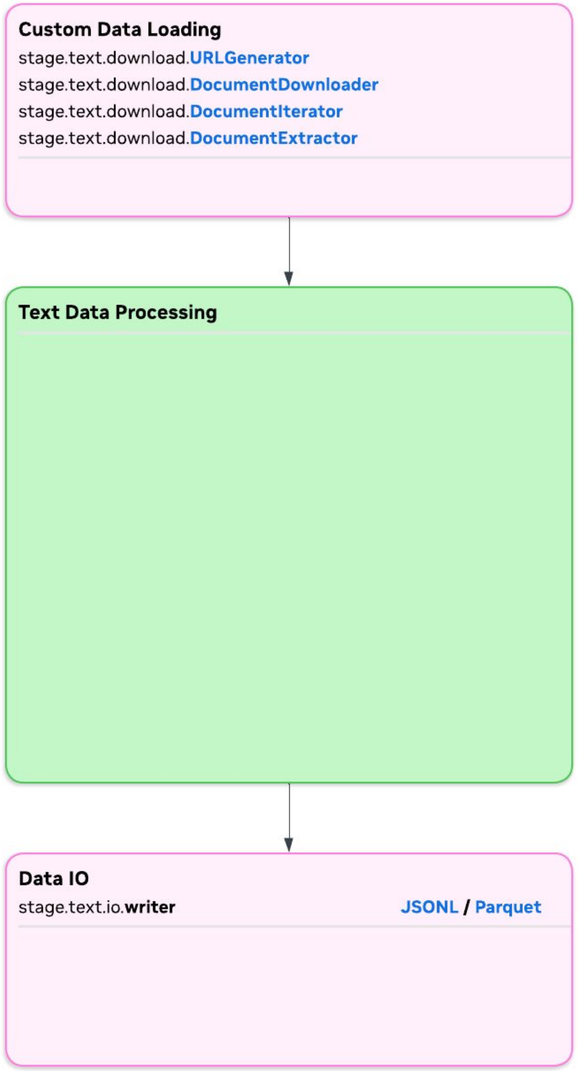

```python
from nemo_curator.stages.text.download import DocumentDownloader

class YourDownloader(DocumentDownloader):
    def _get_output_filename(self, url: str) -> str:
        # Extract filename from URL
        return url.split('/')[-1]

    def _download_to_path(self, url: str, path: str) -> tuple[bool, str | None]:
        # Download logic - return (success_bool, error_message)
        # Use subprocess.run, requests, etc.
```

Advanced Customization



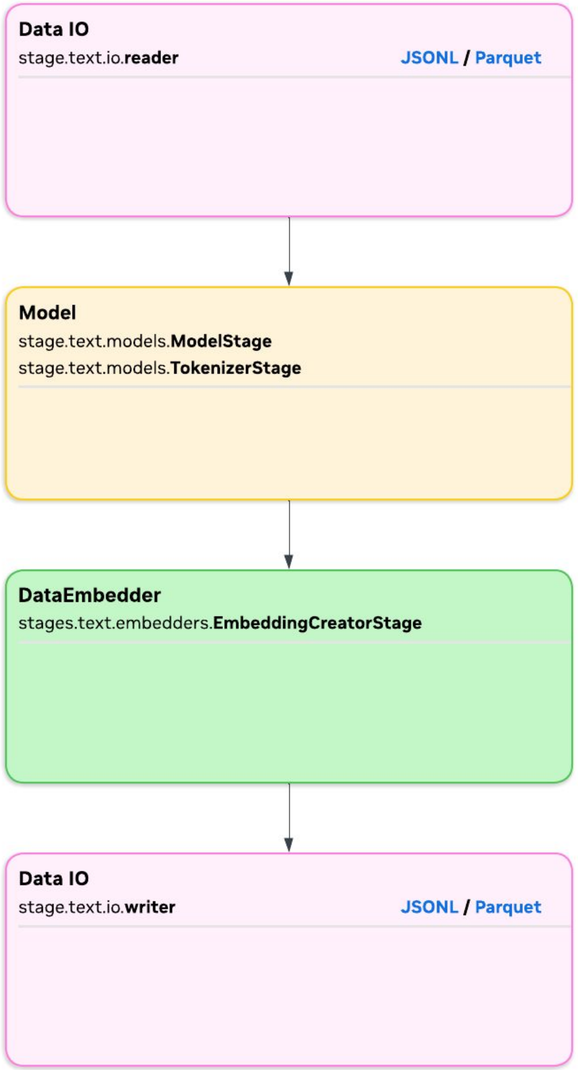

```python
from nemo_curator.stages.text.download import DocumentIterator

class YourIterator(DocumentIterator):
    def iterate(self, file_path: str) -> Iterator[dict[str, Any]]:
        # Parse file and yield record dicts
        yield {"content": "raw_text", "metadata": "value"}

    def output_columns(self) -> list[str]:
        return ["content", "metadata"]
```

Advanced Customization



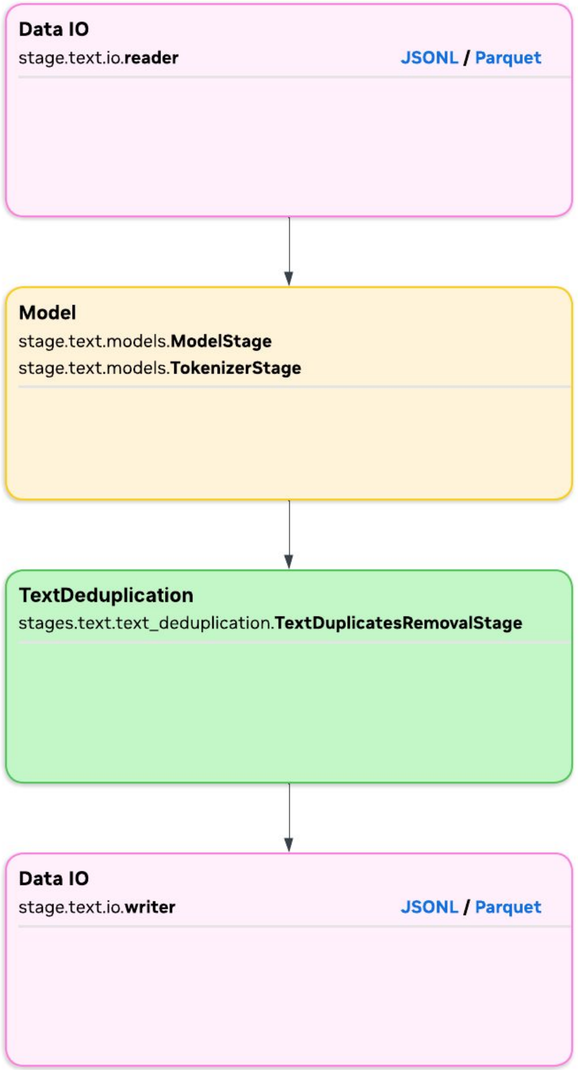

```python
from nemo_curator.stages.text.download import DocumentExtractor

class YourExtractor(DocumentExtractor):
    def extract(self, record: dict[str, str]) -> dict[str, Any] | None:
        # Transform raw record to final format
        return {"text": processed_text, "lang": language}

    def input_columns(self) -> list[str]:
        return ["content", "metadata"]

    def output_columns(self) -> list[str]:
        return ["text", "lang"]
```

Advanced Customization




```python
from nemo_curator.stages.text.embedders import EmbeddingCreatorStage

class YourEmbeddingCreatorStage(EmbeddingCreatorStage):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)

    def execute(self, *args, **kwargs):
        # Logic to create and store embeddings
        pass
```

More Stage




```python
from nemo_curator.stages.text.text_deduplication import TextDuplicatesRemovalStage

class YourTextDuplicatesRemovalStage(TextDuplicatesRemovalStage):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)

    def execute(self, *args, **kwargs):
        # Logic to identify and remove text duplicates
        pass
```

Workflow

```python
from nemo_curator.workflows.text_deduplication import TextDuplicatesRemovalWorkflow
def run_text_deduplication_workflow():
    workflow = TextDuplicatesRemovalWorkflow(
        input_data_dir="/path/to/data",
        output_data_dir="/path/to/output"
    )
    workflow.run()
```

```python
from nemo_curator.workflows.semantic_deduplication import SemanticDeduplicationWorkflow
def run_semantic_deduplication_workflow():
    workflow = SemanticDeduplicationWorkflow(
        input_data_dir="/path/to/data",
        output_data_dir="/path/to/output"
    )
    workflow.run()
```

PDFExtractor

Thank You!

Questions?

NVIDIA